

Підготовка до захисту — «Нічний Дозор»

Документ містить відповіді на три блоки:

1. Питання по коду, які може задати викладач
2. Константи, масиви, MonoBehaviour, Awake і Start
3. Модифікатори та базові поняття C# у проєкті

Актуальна версія: 4 скрипти + карта в сцені SampleScene.

Частина 1. Питання, які може задати викладач

1. Загальна архітектура

П: Зі скількох скриптів складається гра і за що кожен відповідає?

В: 4 файли в Assets/Scripts/:

Файл	Роль
GameConfig.cs	Усі числа: ціни, НР, хвили, шлях ворогів
BuildZone.cs	Зелена клітинка на сцені для башні
Game.cs	Логіка гри, башні, вороги, автозапуск
UIManager.cs	Меню, HUD, екран кінця гри

П: Як гра запускається без ручного налаштування?

В: Unity завантажує сцену SampleScene. На ній вже є об'єкт NightWatch зі скриптами Game і UIManager. Камера і карта (Level) теж на сцені.

П: Де «мозок» гри?

В: Клас Game — золото, хвили, таймер, кліки по карті, НР кристала, перемога/поразка.

П: Де зберігаються всі числа?

В: У GameConfig.cs — константи, масив Waves, масив EnemyPath.

П: Де карта?

В: У сцені SampleScene — об'єкт Level (дорога, зони будівництва, кристал). Код не генерує карту в runtime, а знаходить BuildZone на сцені.

П: Що таке namespace NightWatch?

В: Спільний простір імен — усі класи гри в одному пакеті.

2. Unity і життєвий цикл

П: Чим відрізняються Awake, Start, Update?

В:

- Awake — одразу при створенні (Game.I = this, побудова UI в UIManager)
- Start — один раз перед першим кадром (знайти Level, BuildZone, показати меню)
- Update — кожен кадр (таймер хвили, HUD, атака башен, рух ворогів)

П: Що таке MonoBehaviour?

В: Базовий клас Unity-скрипта; дає Update, Start, transform, gameObject.

П: Що таке Game.I?

В: Singleton — один екземпляр Game на всю гру. У Awake: I = this. Інші класи звертаються через Game.I.

3. Ігровий процес

П: Як гравець починає гру?

В: Меню → кнопка «Ельфи» або «Гноми» → StartWithRace() → скидання золота/НР, показ HUD.

П: Як ставиться башня?

В:

1. Натиснути «Лучник» або «Гармата» (SelectTower)
2. Клікнути зелену клітинку
3. HandleClick → ZoneUnderMouse → Tower.Create

П: Як працює таймер хвили?

В: При старті хвили WaveTimeLeft = GameConfig.WaveSeconds(...). У Update час зменшується. Якщо WaveTimeLeft <= 0 — миттєва поразка (EndGame(false, "■■■■ ■■■■■■!")). Overtime немає.

П: Коли хвиля вважається пройденою?

В: Коли `_waveSpawnDone == true` і `Enemies.Count == 0` → `FinishWave()` → бонусне золото.
Після 5-ї хвили — перемога.

П: Як запускається хвиля?

В: Кнопка «Хвиля» → `StartNextWave()` → корутина `SpawnWave`.

П: Де HP кристала?

В: `Game.CrystalHp`. Ворог, що дійшов до кінця шляху, викликає `DamageCrystal()`.

4. Шлях ворогів

П: Як вороги знають, куди йти?

В: Масив `GameConfig.EnemyPath` — список точок `Vector3`. Ворог у `Update` рухається `MoveTowards` від точки до точки (`_i` — індекс).

П: Чому не BFS?

В: Шлях зафіксований у конфігу. Карта вже намальована в сцені, тому runtime-пошук шляху не потрібен — простіше для захисту.

5. Tower і Enemy

П: Як башня обирає ціль?

В: У `Tower.Update` перебір `Game.I.Enemies` — найближчий живий ворог у радіусі `TowerRange`.

П: Чи є снаряди?

В: Ні. Урон миттєвий — `target.Hit(dmg)` без окремого класу `Projectile`.

П: Чим відрізняються башні?

В: Лучник — дешевший, ближній, швидкий. Гармата — дорожча, дальня, повільніша.

П: Як працюють раси?

В: `GameConfig.ApplyRace`: ельфи +15% швидкість атаки, гноми +20% урон.

6. UI

П: Чому UI створюється кодом?

В: У `UIManager.Awake` → `BuildInterface()` — `Canvas`, кнопки, тексти без `Inspector`.

П: Як підсвічується вибрана башня?

В: `highlightTower()` — зелена кнопка яскравіша, інша тьмяніша.

П: Як не будувати башню при кліку по кнопці?

В: `OverUi()` — `raycast` по `Canvas`; якщо курсор над UI, `HandleClick` виходить.

7. Корутини

П: Що таке корутина?

В: Метод `IEnumerator` з `yield return WaitForSeconds` — спавн ворогів з паузами.

Приклад:

```
IEnumerator SpawnWave(int wave) {  
    ...  
    yield return new WaitForSeconds(GameConfig.SpawnInterval);  
}
```

8. Механіки проєкту (коротко)

Механіка	Де в коді
Вибір раси	<code>UIManager</code> → <code>StartWithRace</code>
2 типи башен	<code>TowerType</code> , кнопки HUD

5 хвиль	GameConfig.WaveCount, масив Waves
Таймер хвили	WaveTimeLeft, поразка при 0
Захист кристала	CrystalHp, DamageCrystal
Карта в сцені	Level + BuildZone на об'єктах

Відповідь на 30 секунд (шаблон)

«Гра — tower defense «Нічний Дозор» на Unity. 4 скрипти: GameConfig — баланс, BuildZone — клітинки на сцені, Game — логіка + башні + вороги, UIManager — інтерфейс. На сцені NightWatch з Game і UIManager, карта в Level. Гравець обирає расу, ставить лучника або гармату на зелені зони, запускає 5 хвиль. Вороги йдуть по EnemyPath. Башні б'ють найближчого ворога без снарядів. Якщо час хвили вийшов або кристал знищено — поразка.»

Частина 2. Константи, масиви та MonoBehaviour

Константи (const)

```
public const int StartGold = 120;
public const int WaveCount = 5;
```

Число задається один раз у `GameConfig` і не змінюється під час гри. Золото гравця — окреме поле `Game.Gold`, яке змінюється.

Масиви

```
public static readonly string[] TowerNames = { "Турецький", "Турецький" };
public static readonly Vector3[] EnemyPath = { ... };
public static readonly (int scout, int fighter, int tank)[] Waves = { ... };
```

- `TowerNames[0]` → «Лучник»
- `Waves[2]` → хвиля 3: 2 скаути, 2 бійці, 1 танк
- `EnemyPath` — точки маршруту ворога

MonoBehaviour у проєкті

Клас	MonoBehaviour?
Game, Tower, Enemy, BuildZone, UIManager	Так
GameConfig, Shape, GameBootstrap	Hi (static)

Awake / Start / Update у грі

```
// Game.cs
void Awake() => I = this;
void Start() { Level, BuildZones, ...; }
void Update() { ... , HUD; }

// Tower.cs
void Update() { ... , ...; }

// Enemy.cs
void Update() { ... EnemyPath; }
```

Частина 3. Модифікатори — шпаргалка

Термін	Одна фраза
public	Видно всім класам
private / _поле	Тільки всередині класу
static	Належить класу, не об'єкту
const	Число-правило, не змінюється
readonly	Задано один раз (масиви імен)
enum	Список варіантів: TowerType, Race
List<T>	Динамічний список ворогів
IEnumerator	Корутина з паузами
Game.I	Єдиний менеджер гри

Проект: «Нічний Дозор» — Tower Defense, Unity, namespace NightWatch, GitHub: Ekzamen_Bozhko.